



空间数据库原理与应用

第十讲：空间数据库索引与查询（二）

任课教师：李晓明

建筑与城市规划学院 城市空间信息工程系

其中部分图片来自互联网和同行专家

上节课回顾

扩展的SQL查询语言主要类型

- 1、查询谓词的扩展
- 2、面向对象的扩展

OpenGIS支持四个子空间几何类型： Point; Curve; Surface; GeometryCollection

基本函数	SpatialReference	几何体的坐标系
	Envelope	几何体最小外接矩形
	Export	其他形式的几何体
	IsEmpty	判断几何体是否为空
	IsSimple	判断几何体是否自交
	Boundary	几何体的边界
拓扑/集合运算	Equal	判断两个几何体是否相等
	DisJoint	判断几何体是否相接
	Intersect	判断几何体是否相交
	Touch	判断几何体是否相邻
	Cross	判断线与面是否相交
	Within	判断是否被包含
	Contains	判断是否包含
	Overlap	判断是否有重叠
空间分析	Distance	几何体之间最短距离
	Buffer	几何体缓冲区
	ConvexHull	几何体最小凸包
	Intersection	两个几何体的交集
	Union	两个几何体的并集
	Difference	几何体的差集
	SymmDiff	两个几何体的不相交部分

上节课回顾

操作2.2曼哈顿 (Manhattan) 行政区的面积是多少 英 亩？（提示：nyc_census_blocks 和 nyc_neighborhoods 中都有 boroname - rorough name - 行政区名）

```
SELECT Sum(ST_Area(geom)) / 4047
FROM nyc_neighborhoods
WHERE boroname = 'Manhattan';
```

或:

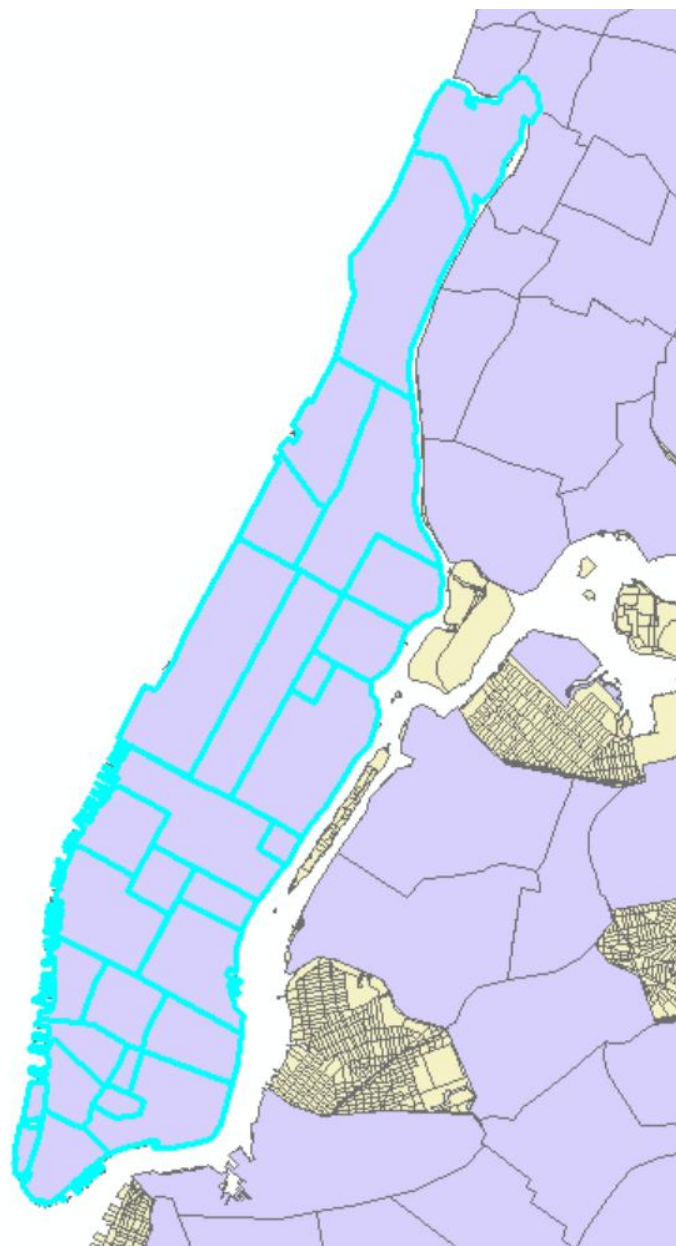
```
SELECT Sum(ST_Area(geom)) / 4047
FROM nyc_census_blocks
WHERE boroname = 'Manhattan';
```

```
geo_test=# SELECT Sum(ST_Area(geom)) / 4047
geo_test=# FROM nyc_neighborhoods
geo_test=# WHERE boroname = 'Manhattan';
?column?
```

```
13965.3201223912
(1 行记录)
```

```
geo_test=# SELECT Sum(ST_Area(geom)) / 4047
geo_test=# FROM nyc_census_blocks
geo_test=# WHERE boroname = 'Manhattan';
?column?
```

```
14601.3987215548
(1 行记录)
```



目录

CONTENTS

04

PostGIS栅格数据查询

05

空间查询执行

06

空间查询优化

目录

CONTENTS

4

PostGIS栅格数据查询

4.1 posgGIS管理栅格数据

4.2 PostGIS导入栅格数据

4.3 PostGIS导出栅格数据

4.4 PostGIS栅格函数

4.1 PostGIS管理栅格数据

在PostGIS中，`raster`数据类型用于存储和操作栅格数据。其定义是一个可以容纳多波段栅格图像的数据结构，每个波段可以有不同的像素深度和数据类型（例如无符号整数、有符号整数、浮点数等）。

`raster`数据类型不仅存储像素值，还存储有关栅格的空间参考、像素大小、偏移量和旋转等元数据信息。

如何管理栅格数据：

1.加载扩展：首先，需要在PostgreSQL数据库中加载PostGIS及其栅格相关的扩展。这可以通过运行如下SQL命令完成：

- `CREATE EXTENSION IF NOT EXISTS postgis;`

- 创建表：**创建一个包含`raster`类型字段的表来存储栅格数据。例如，创建一个名为`my_raster_table`的表，其中包含一个名为`geom`的栅格列：

```
CREATE TABLE my_raster_table ( id SERIAL PRIMARY KEY, geom raster, name  
VARCHAR(100) );
```

4.1 PostGIS管理栅格数据

导入栅格数据: 使用ST_AddRasterConstraints和其他相关函数来导入栅格数据并设置约束条件, 确保数据的完整性。例如, 导入一个.tif文件到geom列:

sql

```
INSERT INTO my_raster_table (geom)
```

```
•VALUES (ST_SetSRID(ST_RasterFromGDAL('/path/to/myfile.tif'), <SRID>));
```

•设置约束: 可以对栅格列设置约束来管理数据的属性, 比如空间参考系统(SRID)、像素大小、对齐方式等, 以保证数据的一致性和高效查询:

sql

```
SELECT ST_AddRasterConstraints('my_raster_table', 'geom', 'PIXEL_TYPES', TRUE);
```

4.1 PostGIS管理栅格数据

导入栅格数据:

假设你有一个名为example.tif的栅格文件，你想将其导入到刚创建的表中。首先，你需要使用raster2pgsql工具生成一个SQL脚本，然后执行这个脚本来导入数据。以下是一个简化的命令行示例：

```
raster2pgsql -s 4326 -l -C example.tif public.myraster raster_column > load_raster.sql  
psql -d mydatabase -f load_raster.sql
```

这里，-s 4326指定了空间参考ID为EPSG:4326，-l表示添加索引，-C表示添加常规的栅格约束。

4.1 PostGIS管理栅格数据

查询和分析: 利用PostGIS提供的栅格函数和操作符进行空间查询和分析，比如裁剪、重采样、统计分析等。例如，获取某个栅格区域的像素值总和：

sql

- SELECT** ST_Summary(rast, 1) **FROM** my_raster_table **WHERE** ST_Intersects(rast, ...);

获取栅格的宽度和高度：

SELECT ST_Width(raster_column), ST_Height(raster_column) **FROM** myraster **WHERE** rid = 1;

或者，查询某个栅格区域内的平均像素值：

SELECT STixelAverage(raster_column, 1) **FROM** myraster **WHERE**
ST_Intersects(raster_column, ST_GeomFromText('POLYGON(...))));

•**导出数据:** 可以使用ST_AsGDALRaster或类似函数将栅格数据导出到文件系统，或者通过其他PostGIS函数转换为不同的栅格格式或矢量数据。

4.2 PostGIS导入栅格数据

USAGE: raster2pgsql [<options>] <raster>[<raster>[...]] [[<schema>.]<table>]

OPTIONS:

(c|a|d|p) 下面这些选项互斥:

- c 创建新表, 并把栅格数据导入到表中, 这是默认模式
- a 追加栅格数据到一个已经存在的表
- d 先drop掉表, 然后创建新表, 并把栅格数据导入到表中
- p 准备模式, 只创建表, 不导入数据.

栅格处理过程: 把适当的约束条件注册 (写入) 栅格系统表中。

- C 用于栅格约束注册——比如栅格的srid, pixelsize 元数据信息等等, 保证栅格在视图raster_columns能够合适地注册。
- x 让设置最大边界约束失效, 只有在-C参数同时使用时候才能使用这个参数。
- r 为规则的块设置约束 (空间唯一性约束和覆盖瓦片)。只有在同时使用-C 参数时, 才能使用这个参数。

栅格处理参数: 用于操作输入栅格数据集的可选参数。

- s 表示<SRID>: 指定输出栅格使用指定的SRID。如果没有提供或者提供值为0, 程序会检查栅格的元数据 (比如栅格文件) 以便决定一个合适的SRID。
- b 从栅格要抽取的波段位置 (下标从1开始)。想要抽取不止1个波段的话, 使用逗号, 分隔波段数字。如果没有指定, 所有栅格的波段都会被抽取出来。
- t TILE_SIZE: 瓦片大小。把每一个表的行表示的栅格切割成瓦片。TILE_SIZE被表示成WIDTHxHEIGHT的形式, 或者设置成值为“auto”来让加载程序使用第一个栅格参数计算合适的瓦片大小, 然后把这个瓦片大小应用于所有的栅格
- R, --register 把栅格以文件数据形式注册 (栅格数据保存在数据库之外的文件系统中)。只有栅格的元数据和文件路径保存在数据库中 (没有保存像素的数据)
- I OVERVIEW_FACTOR: 创建栅格的概览信息。如果factor值不只一个, 使用逗号, 进行分隔。创建的概览数据存储在数据库中, 并且不受参数-R影响。注意你生成的SQL文件会包含存储数据的主表和概览表。
- N NODATA: 指定没有NODATA值的波段的NODATA值。

4.2 PostGIS导入栅格数据

USAGE: raster2pgsql [<options>] <raster>[<raster>[...]] [[<schema>.]<table>]

OPTIONS:

用于操作数据库对象的可选参数

- q : 把PostgreSQL的id用引号包起来。
- f COLUMN :指定目标的栅格列名，默认是 'rast'。
- F : 创建一个以文件名为列名的列
- I : 在栅格列上创建一个GiST索引
- M Vacuum analyze栅格表
- T tablespace : 指定新表的表空间。注意如果索引（包括primary key）依然会使用默认的表空，除非使用-X参数标识。
- X tablespace : 指定表的索引表空间。如果使用了参数-I，那么这个参数可以应用于primary key和空间索引。
- Y : 使用copy语句而不是insert 语句。

```
"C:\Program Files\PostgreSQL\11\bin\raster2pgsql.exe"-s 4326 -I -C -M
D:/Data/world_raster/wsiearth.tif -F -t 256x256 public.worldraster | "C:\Program
Files\PostgreSQL\11\bin\psql.exe " -h localhost -p 5432 -U postgres -d geo_test -W
```

```
C:\Users\lxmflyx1>"C:\Program Files\PostgreSQL\11\bin\raster2pgsql.exe" -s 4326 -I -C -M D:/Data/world_raster/wsiearth.tif \
if -F -t 256x256 public.worldraster | "C:\Program Files\PostgreSQL\11\bin\psql.exe" -h localhost -p 5432 -U postgres -d \
geo_test -W
命令: Processing 1/1: D:/Data/world_raster/wsiearth.tif

BEGIN
CREATE TABLE
INSERT 0 1
INSERT 0 1
INSERT 0 1
INSERT 0 1
INSERT 0 1
INSERT 0 1
INSERT 0 1
INSERT 0 1
INSERT 0 1
INSERT 0 1
INSERT 0 1
ANALYZE
    娉几剩: Adding SRID constraint
    娉几剩: Adding scale-X constraint
    娉几剩: Adding scale-Y constraint
    娉几剩: Adding blocksize-X constraint
    娉几剩: Adding blocksize-Y constraint
    娉几剩: Adding alignment constraint
    娉几剩: Adding number of bands constraint
    娉几剩: Adding pixel type constraint
    娉几剩: Adding nodata value constraint
    娉几剩: Adding out-of-database constraint
    娉几剩: Adding maximum extent constraint
addrasterconstraints
-----
t
(1 行记录)

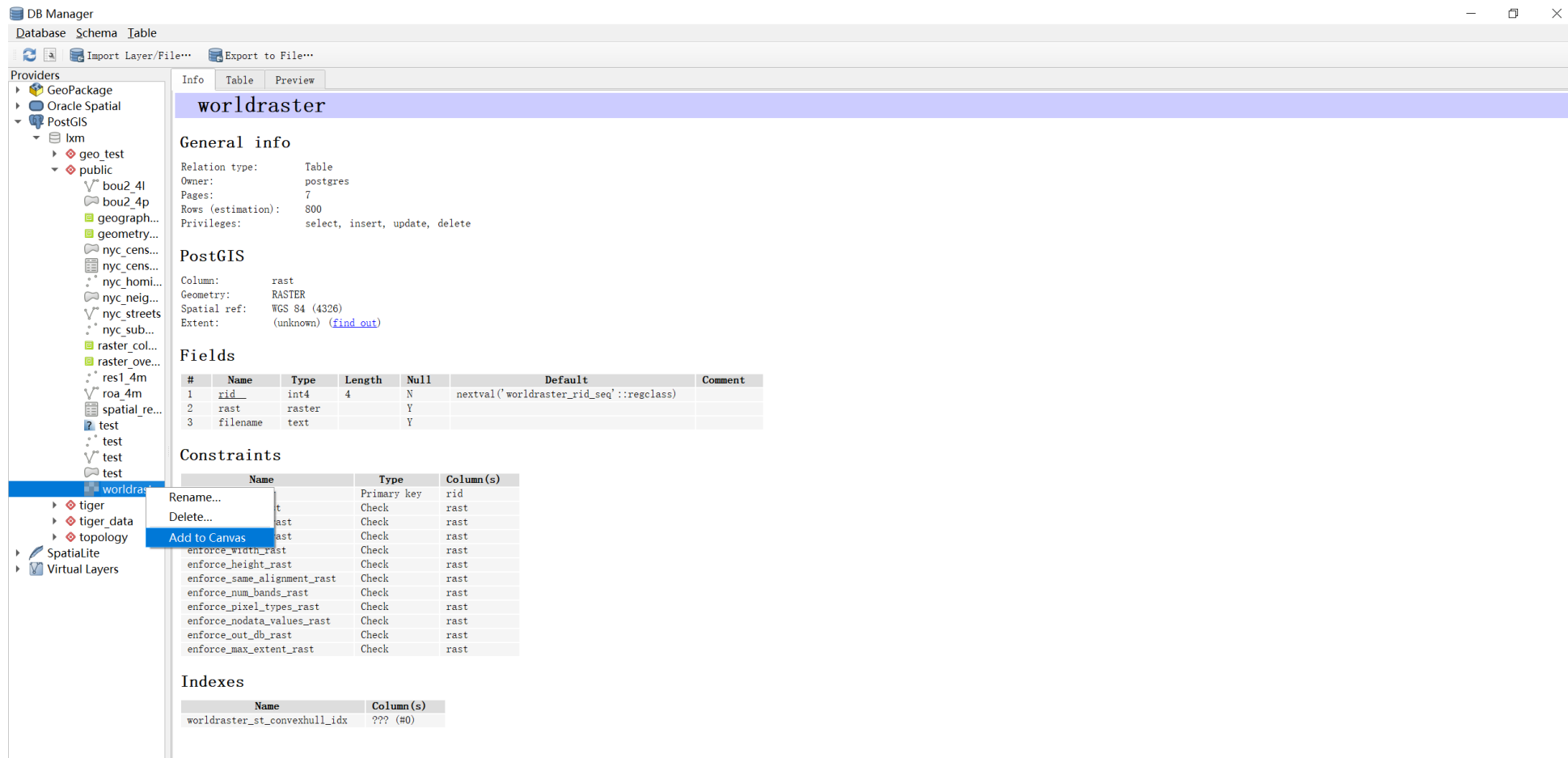
COMMIT
VACUUM
```

```
<!-- 以单个文件方式存储到PostGIS数据库中 -->
```

4.2 PostGIS导入栅格数据

如何在QGIS中查看

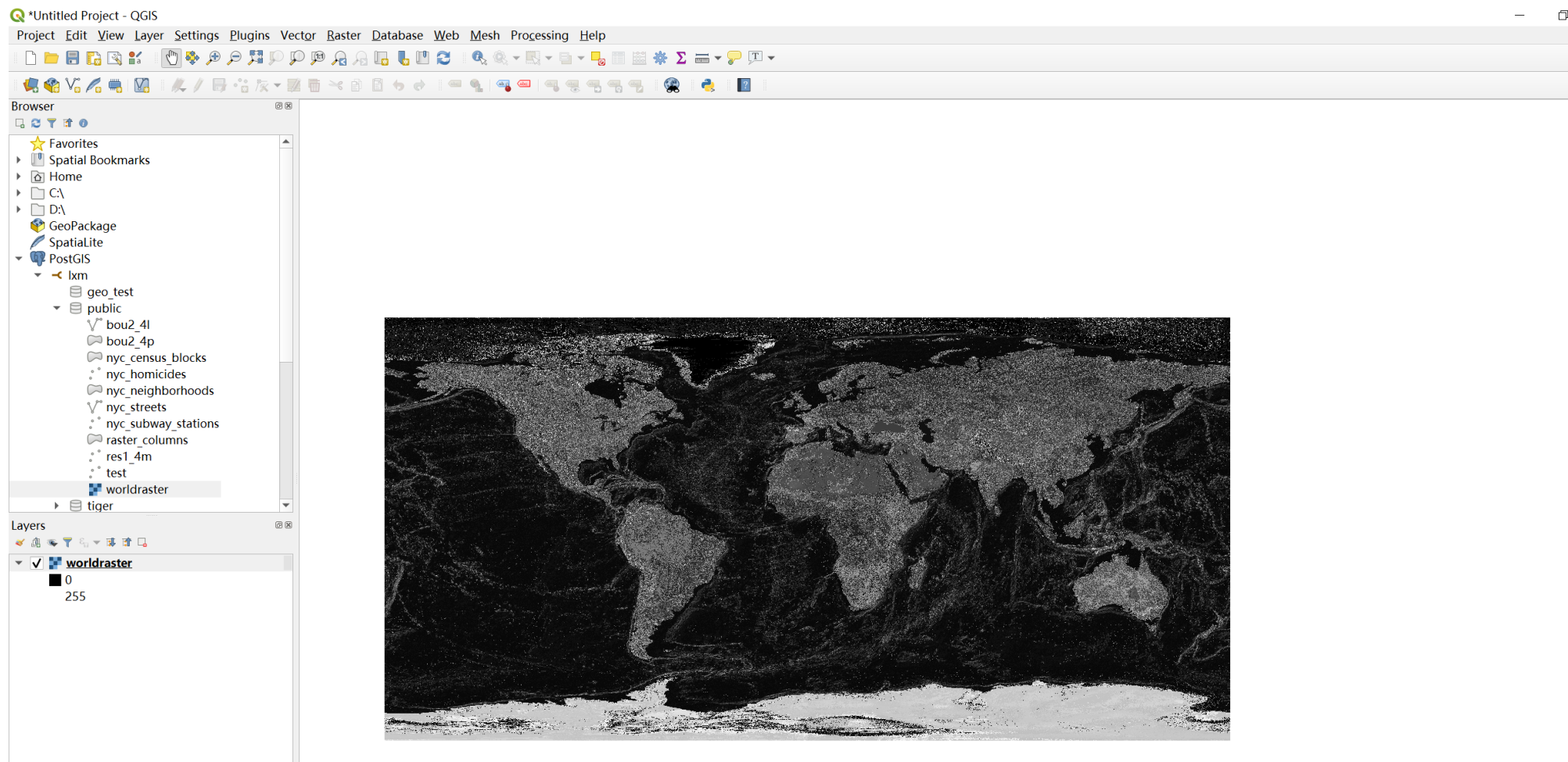
QGIS中无法直接显示PostGIS空间数据库中的影像，要找到影像显示打开 “Database” -> “DB Manager”，右击——> “Add to Canvas”



4.2 PostGIS导入栅格数据

如何在QGIS中查看

QGIS中无法直接显示PostGIS空间数据库中的影像，要找到影像显示打开 “Database” -> “DB Manager”，右击——> “Add to Canvas”



4.2 PostGIS导入栅格数据

使用PostGIS栅格函数来创建栅格

1. 创建一个带有栅格记录的栅格列的表可以用下面的完成：

```
CREATE TABLE myrasters(rid serial primary key, rast raster);
```

2. 如果创建的栅格不依赖于其他栅格，可以使用函数：ST_MakeEmptyRaster，接着使用ST_AddBand

也可以使用geometry对象来创建栅格，需要使用函数ST_AsRaster，可能还需要和其他函数比如ST_Union 或 ST_MapAlgebraFct 或者其他地图代数系列函数联合使用

3.一旦完成了栅格表初始化，你需要在你的栅格列上创建一个空间索引，使用如下语法：

```
CREATE INDEX myrasters_rast_st_convexhull_idx ON myrasters USING gist( ST_ConvexHull( rast) );
```

注意这里函数ST_ConvexHull 的使用，因为大多数栅格操作符都是基于栅格的凸包。

4. 使用函数AddRasterConstraints添加约束

4.2 PostGIS导入栅格数据

栅格系统目录

两个栅格视图都利用嵌入在栅格约束中的信息进行展示。因此这两个视图总是与表中的栅格数据保持一致性兼容，因为表中强加了约束。

1. `raster_columns` 这个视图记录了包含你数据库中所有的栅格表的列。
2. `raster_overviews` 这个视图记录了包含你的数据库中所有的栅格表的列，并以细粒度表的方式概述。这种类型的表在你使用栅格加载程序并加上-l参数时候将会生成。

4.3 PostGIS导出栅格数据

针对已经通过PostGIS导入到PostgreSQL中的栅格数据，给定经纬度范围，实现栅格数据的导出。
查询SQL语句如下：

```
SELECT ST_Union(ST_Clip(rast,geom)) AS rast FROM public. worldraster CROSS JOIN ST_MakeEnvelope(90,60,120,45,4326) As  
geom WHERE ST_Intersects(rast,geom);
```

其中PostGIS函数的含义如下：

ST_MakeEnvelope：函数用于构造一个矩形范围，其参数分别是最小X值，最小Y值，最大X值，最大Y值和坐标系代码
ST_Intersects：函数用于选择出与geom矩形相交的栅格Tiles
ST_Clip：函数用于将选择出来的Tiles进行裁剪，得到geom范围的数据
ST_Union：函数用于聚合选择出来的数据为一个整体

导出TIF格式的SQL语句如下：

```
SELECT ST_AsTIFF(rast, 'LZW') FROM (SELECT ST_Union(ST_Clip(rast,geom)) AS rast FROM worldraster CROSS JOIN  
ST_MakeEnvelope(90,60,120,45,4326) As geom WHERE ST_Intersects(rast,geom)) AS rasttiff;
```

ST_MakeEnvelope：函数用于构造一个矩形范围，其参数分别是最小X值，最小Y值，最大X值，最大Y值和坐标系代码
ST_Intersects：函数用于选择出与geom矩形相交的栅格Tiles
ST_Clip：函数用于将选择出来的Tiles进行裁剪，得到geom范围的数据
ST_Union：函数用于聚合选择出来的数据为一个整体

4.3 PostGIS导出栅格数据

使用Python的Psycopg库连接PostgreSQL数据库，进行查询并导出最终的结果

```
# -*- coding: utf-8 -*-
```

```
import psycopg2
```

```
# Connect to an existing database
```

```
conn = psycopg2.connect('host=localhost port=5432 user=postgres password=password dbname=geo_test')
```

```
# Open a cursor to perform database operations
```

```
cur = conn.cursor()
```

```
# Execute SQL query
```

```
cur.execute("SELECT ST_AsTIFF(rast, 'LZW') FROM (SELECT ST_Union(ST_Clip(rast,geom)) AS rast FROM worldraster CROSS JOIN  
ST_MakeEnvelope(90,60,120,45,4326) As geom WHERE ST_Intersects(rast,geom)) AS rasttiff;")
```

```
# Fetch data as Python objects
```

```
rasttiff = cur.fetchone()
```

```
# Write data to file
```

```
if rasttiff is not None:
```

```
    open('/home/theone/Desktop/wsiearth.tif', 'wb').write(str(rasttiff[0]))
```

```
# Close communication with the database
```

```
cur.close()
```

```
conn.close()
```

4.4 PostGIS栅格函数

ST_ValueCount提取函数

该函数返回一个记录集，包括像素值和指定栅格（或栅格覆盖）的指定波段的像素值在一个值集合内的像素个数。如果没有指定波段，那么默认是波段1。默认也不统计值为NODATA的像素。像素值如果不是整数，那么像素值会进行round四舍五入处理得到一个最接近的整数值。

具体应用如下：

-- 查询指定范围内栅格，指定波段，指定像素值的数量

```
SELECT rid, ST_ValueCount(rast,2,100) As count  
FROM test  
WHERE ST_Intersects(rast,  
ST_GeomFromText('POLYGON((224486 892151,224486 892200,224706  
892200,224706  
892151,224486 892151))',26986)  
);
```

4.4 PostGIS栅格函数

http://postgis.net/docs/RT_reference.html

Raster Management

- AddRasterConstraints — 为已加载的栅格表中特定列添加栅格约束，以限制空间参考、缩放、块大小、对齐、波段数、波段类型以及一个标志来表示栅格列是否为规则块。表必须已加载数据，以便推断约束条件。如果成功设置约束条件，则返回true；否则，将发出通知。
- DropRasterConstraints — 删除引用栅格表列的PostGIS栅格约束。在需要重新加载数据或更新栅格列数据时非常有用。
- AddOverviewConstraints — 标记一个栅格列作为另一个栅格的概览。
- DropOverviewConstraints — 取消标记一个栅格列作为另一个栅格的概览。
- PostGIS_GDAL_Version — 报告PostGIS使用的GDAL库的版本。
- PostGIS_Raster_Lib_Build_Date — 报告完整的栅格库构建日期。
- PostGIS_Raster_Lib_Version — 报告完整的栅格版本和构建配置信息。
- ST_GDALDrivers — 返回PostGIS通过GDAL支持的栅格格式列表。只有can_write=True的格式才能被ST_AsGDALRaster使用。
- UpdateRasterSRID — 更改用户指定列和表中所有栅格的数据坐标参考系统（SRID）。
- ST_CreateOverview — 创建给定栅格覆盖范围的一个降低分辨率的版本。

4.4 PostGIS栅格函数

Raster Constructors

- ST_AddBand — 返回一个新的栅格，其中在指定索引位置添加了给定类型的新的波段，并可设定初始值。如果没有指定索引，波段将被添加到末尾。
- ST_AsRaster — 将PostGIS几何对象转换为PostGIS栅格。
- ST_Band — 返回现有栅格的一个或多个波段作为新栅格。这对于根据现有栅格构建新栅格非常有用。
- ST_MakeEmptyCoverage — 用空的栅格瓦片网格覆盖地理参照区域。
- ST_MakeEmptyRaster — 返回一个空的栅格（没有波段），具有给定的尺寸（宽度和高度）、左上角X和Y坐标、像素大小和旋转（scalex, scaley, skewx & skewy）以及参考系统（srid）。如果传入了一个栅格，将返回一个具有相同大小、对齐方式和SRID的新栅格。如果不指定srid，空间参考将设置为未知（0）。
- ST_Tile — 根据输出栅格所需尺寸，将输入栅格分割并返回一组栅格。
- ST_Retile — 从任意瓦片化的栅格覆盖中返回一组配置好的瓦片。
- ST_FromGDALRaster — 从受支持的GDAL栅格文件中返回一个栅格。

4.4 PostGIS栅格函数

Raster Accessors

ST_GeoReference — 返回栅格的地理参考元数据，格式为GDAL或ESRI格式，常见于世界文件中。默认为GDAL格式。

ST_Height — 返回栅格的高度（以像素计）。

ST_IsEmpty — 如果栅格为空（宽度=0且高度=0），则返回true。否则，返回false。

ST_MemSize — 返回栅格所占的空间量（以字节计）。

ST_MetaData — 返回关于栅格对象的基本元数据，如像素大小、旋转（偏斜）、左上角坐标等。

ST_NumBands — 返回栅格对象中的波段数量。

ST_PixelHeight — 返回栅格在空间参考系统中的像素高度（几何单位）。

ST_PixelWidth — 返回栅格在空间参考系统中的像素宽度（几何单位）。

ST_ScaleX — 返回坐标参考系统单位中的像素宽度的X分量。

ST_ScaleY — 返回坐标参考系统单位中的像素高度的Y分量。

ST_RasterToWorldCoord — 给定列和行，返回栅格左上角的几何X和Y坐标（经度和纬度）。列和行编号从1开始。

ST_RasterToWorldCoordX — 给定列和行，返回栅格左上角的几何X坐标。列和行编号从1开始。

ST_RasterToWorldCoordY — 给定列和行，返回栅格左上角的几何Y坐标。列和行编号从1开始。

ST_Rotation — 返回栅格的旋转角度（以弧度计）。

ST_SkewX — 返回地理参考的X偏斜（或旋转参数）。

ST_SkewY — 返回地理参考的Y偏斜（或旋转参数）。

ST_SRID — 返回栅格在spatial_ref_sys表中定义的空间参考标识符。

ST_Summary — 返回栅格内容的文本摘要。

ST_UpperLeftX — 返回栅格在投影空间参考中的左上X坐标。

ST_UpperLeftY — 返回栅格在投影空间参考中的左上Y坐标。

ST_Width — 返回栅格的宽度（以像素计）。

ST_WorldToRasterCoord — 给定几何X和Y坐标（经纬度）或点几何（在栅格的空间参考坐标系中表示），返回栅格的左上角作为列和行。

ST_WorldToRasterCoordX — 返回栅格中点几何（pt）或世界坐标系中X和Y坐标（xw, yw）表示的列。

ST_WorldToRasterCoordY — 返回栅格中点几何（pt）或世界坐标系中X和Y坐标（xw, yw）表示的行。

4.4 PostGIS栅格函数

ST_Clip - 用于裁剪栅格数据到指定的几何区域内。

- **SELECT** ST_Clip(rast, geom) **FROM** myraster, mygeomtable **WHERE** myraster.rid = mygeomtable.gid;

- **ST_Reproject** - 重投影栅格数据到新的坐标系。

- **SELECT** ST_Reproject(rast, 3857) **FROM** myraster **WHERE** rid = 1;

- **ST_Resample** - 改变栅格的分辨率或像素尺寸。

- **SELECT** ST_Resample(rast, 10, 10) **FROM** myraster **WHERE** rid = 1;

- **ST_MapAlgebra** - 执行栅格代数运算，如加、减、乘、除等。

- **SELECT** ST_MapAlgebra(rast1, rast2, '[rast1]+[rast2]') **FROM** myraster1, myraster2 **WHERE** myraster1.rid = myraster2.rid;

- **ST_Band** - 从多波段栅格中提取单个波段。

- **SELECT** ST_Band(rast, 1) **FROM** mymultibandraster **WHERE** rid = 1;

4.4 PostGIS栅格函数

ST_SummaryStats - 计算栅格数据的统计摘要，如最小值、最大值、平均值等。

- `SELECT ST_SummaryStats(rast, 1, TRUE) FROM myraster WHERE rid = 1;`

ST_Histogram - 计算栅格数据的直方图。

- `SELECT ST_Histogram(rast, 1, 10, 0, 255) FROM myraster WHERE rid = 1;`

ST_Union - 合并多个栅格数据集。

- `SELECT ST_Union(rast) FROM myraster WHERE condition;`

ST_AsPNG - 将栅格数据转换为PNG格式的二进制字符串。

- `SELECT ST_AsPNG(rast) FROM myraster WHERE rid = 1;`

ST_Difference - 计算两个栅格之间的差异。

- `SELECT ST_Difference(rast1, rast2) FROM myraster1, myraster2 WHERE myraster1.rid = myraster2.rid;`

目录

CONTENTS

5

空间查询执行

5.1 查询语句执行过程

5.2 数据库查询处理过程

5.3 空间查询执行过程

5.1 查询语句执行过程

一 连接

1, 客户端把语句发给服务器端执行

1.1客户端发起一条Query请求, 监听客户端的 '连接管理模块' 接收请求

1.2将请求转发到 '连接进/线程模块'

1.3调用 '用户模块' 来进行授权检查

1.4通过检查后, '连接进/线程模块' 从 '线程连接池' 中取出空闲的被缓存的连接线程和客户端请求对接, 如果失败则创建一个新的连接请求

二 处理

2, 语句解析

当客户端把 SQL 语句传送到服务器后,服务器进程会对该语句进行解析

2.1, 查询高速缓存(library cache)。

2.2, 语句合法性检查(data dict cache)。

2.3, 语言含义检查(data dict cache)。

2.4, 获得对象解析锁(control structer)。

2.5, 数据访问权限的核对(data dict cache)。

2.6, 确定最佳执行计划?。

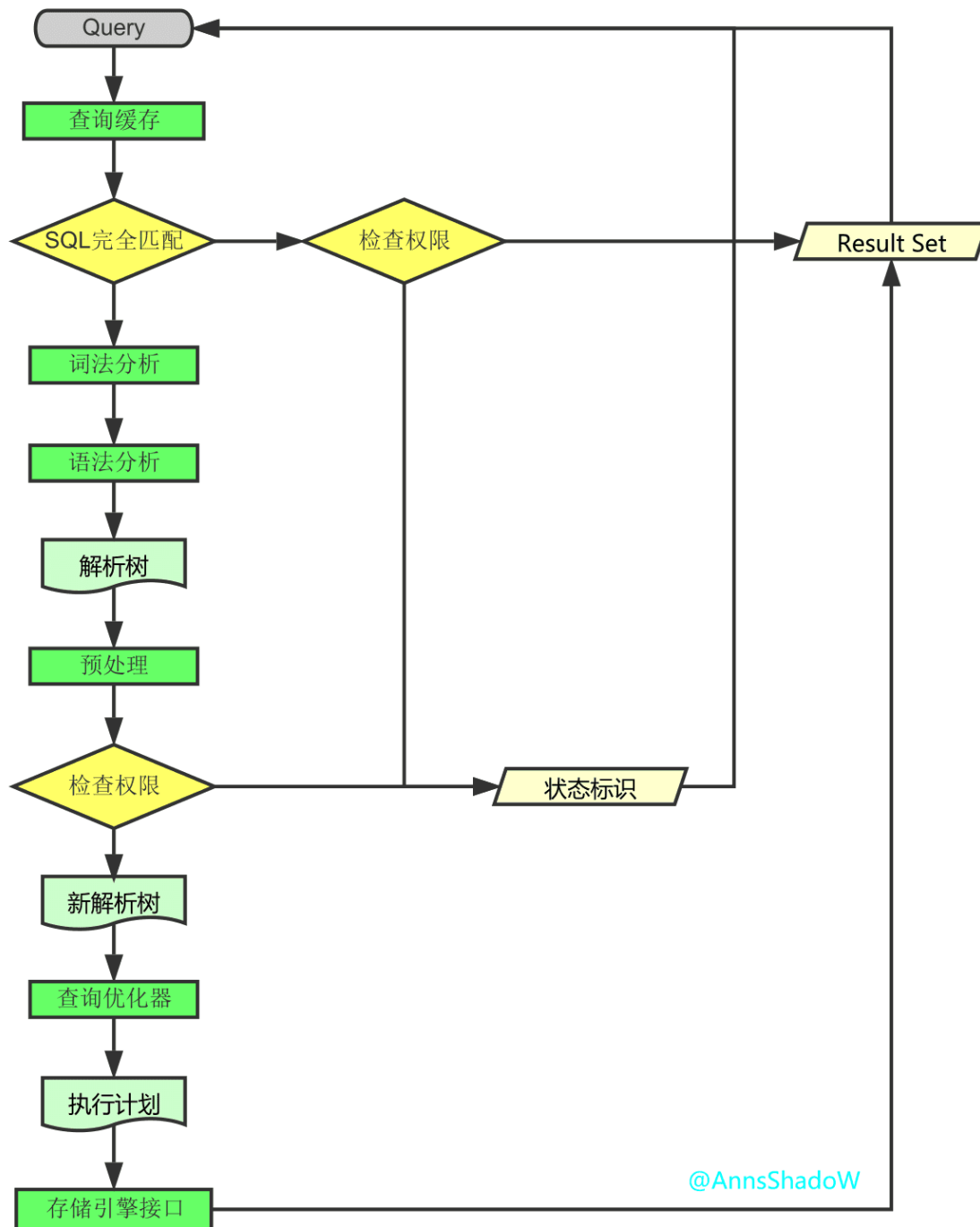
三 提取结果

3.结果

3.1Query请求完成后, 将结果集返回给 '连接进/线程模块'

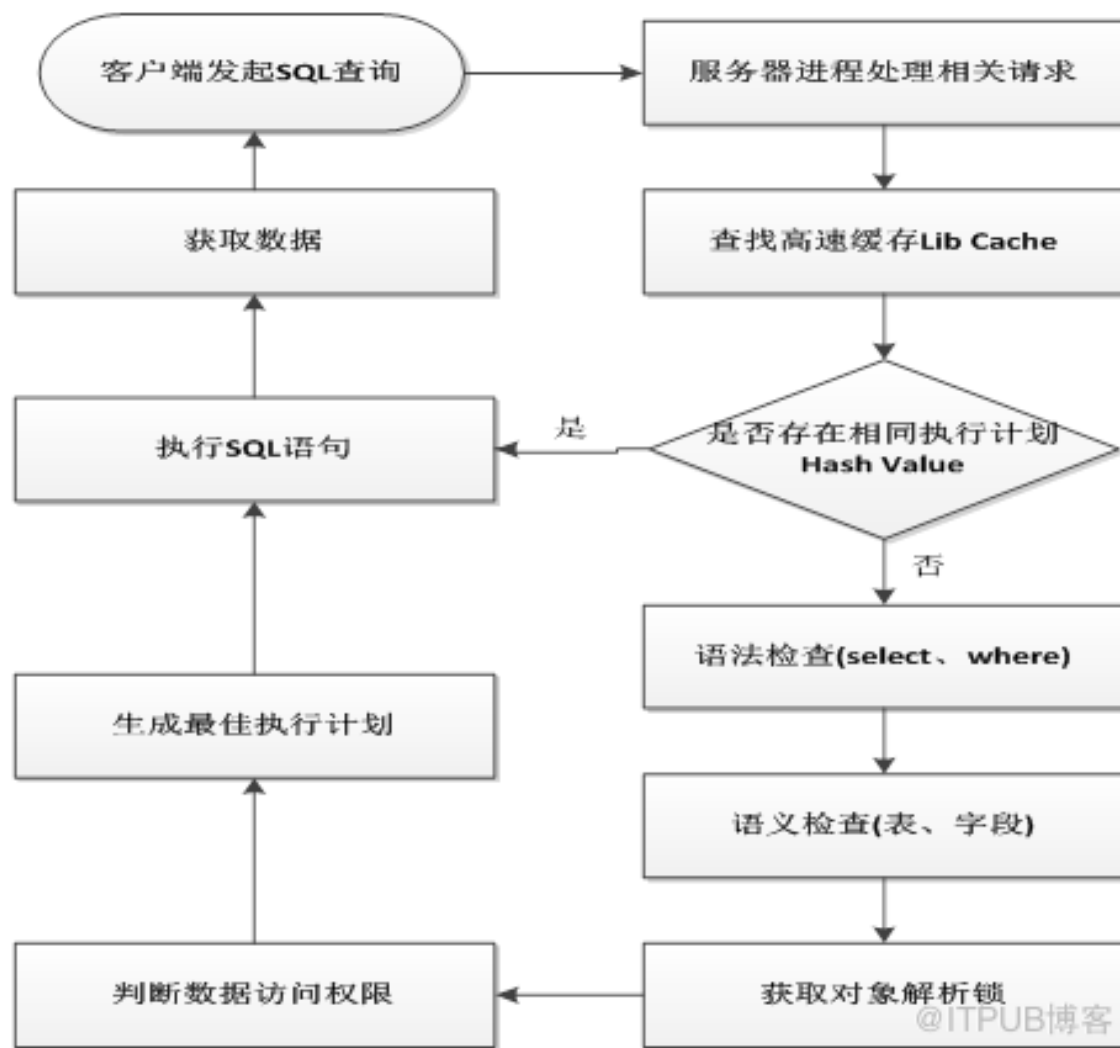
3.2返回的也可以是相应的状态标识, 如成功或失败等

3.3 '连接进/线程模块' 进行后续的清理工作, 并继续等待请求或断开与客户端的连接



@AnnsShadoW

5.1 查询语句执行过程

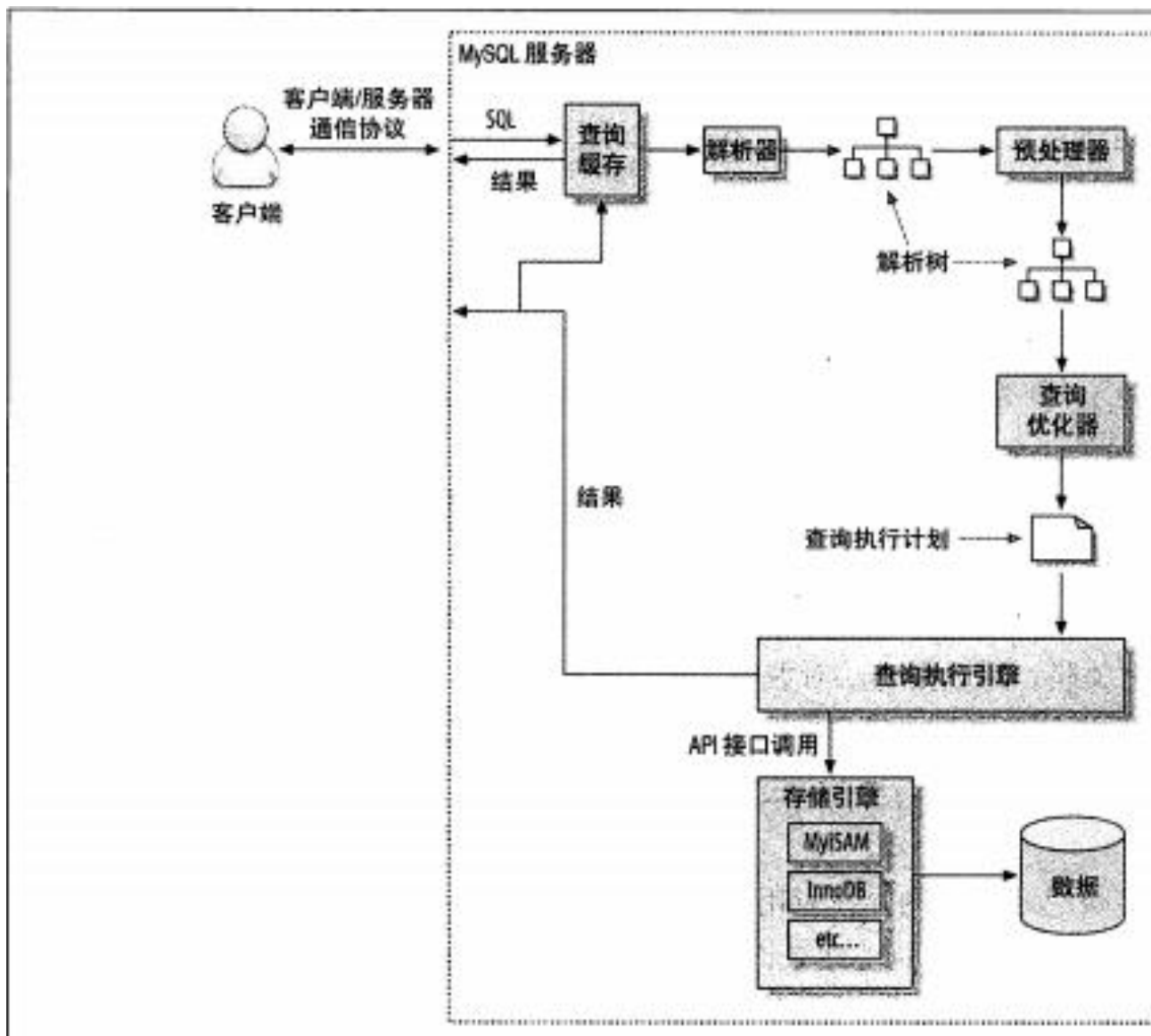


Oracle数据库SQL语句执行过程

5.1 查询语句执行过程

MySQL中的SQL语句执行过程

- 客户端发送一条查询给服务器;
- 服务器先检查查询缓存, 如果命中了缓存, 则立刻返回存储在缓存中的结果。否则进入下一阶段。
- 服务器段进行SQL解析、预处理, 在优化器生成对应的执行计划;
- mysql根据优化器生成的执行计划, 调用存储引擎的API来执行查询。
- 将结果返回给客户端。



5.1 查询语句执行过程

SQL语句的执行顺序

- (8)SELECT (9)DISTINCT (11)<Top Num> <column>
(1)FROM [left_table]
(3)<join_type> JOIN <right_table>
(2) ON <join_condition>
(4)WHERE <where_condition>
(5)GROUP BY <group_by_list>
(6)WITH <CUBE | RollUP>
(7)HAVING <having_condition>
(10)ORDER BY <order_by_list>

<https://blog.csdn.net/dxm809>

5.2 数据库查询处理过程

查询语句

查询解析器

```
SELECT Sn
FROM S, SC
WHERE S.Sno=SC.Sno
AND SC.Cno='c02';
```

语法分析树

查询预处理器

数据字典

关系代数查询树

查询优化器

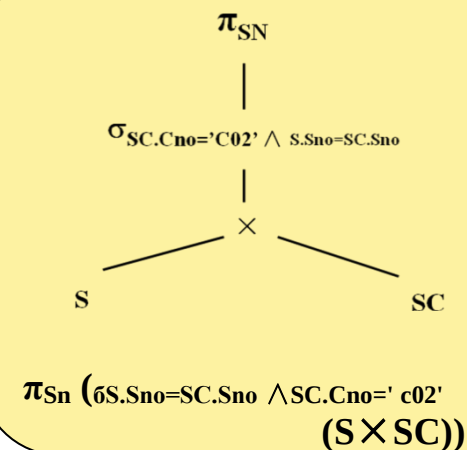
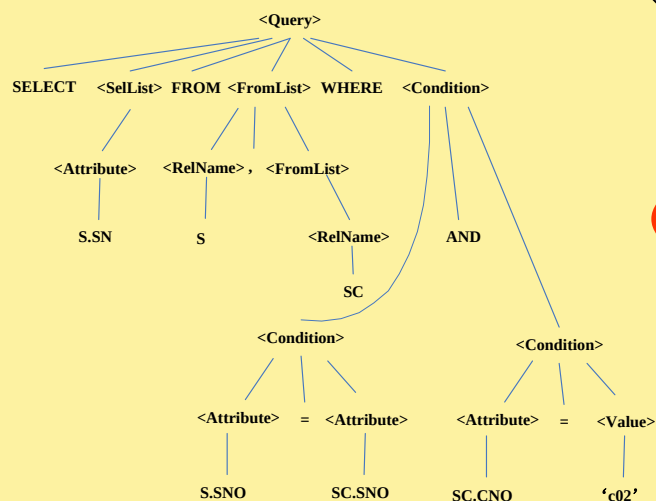
关系代数优化查询树

查询代码生成器

查询计划的执行代码

执行引擎

执行结果



5.2 数据库查询处理过程

查询分析与预处理

- 查询分析
 - 接受类似SQL这样的高级查询语言表示的查询，并进行词法分析和语法分析。

5.2 数据库查询处理过程

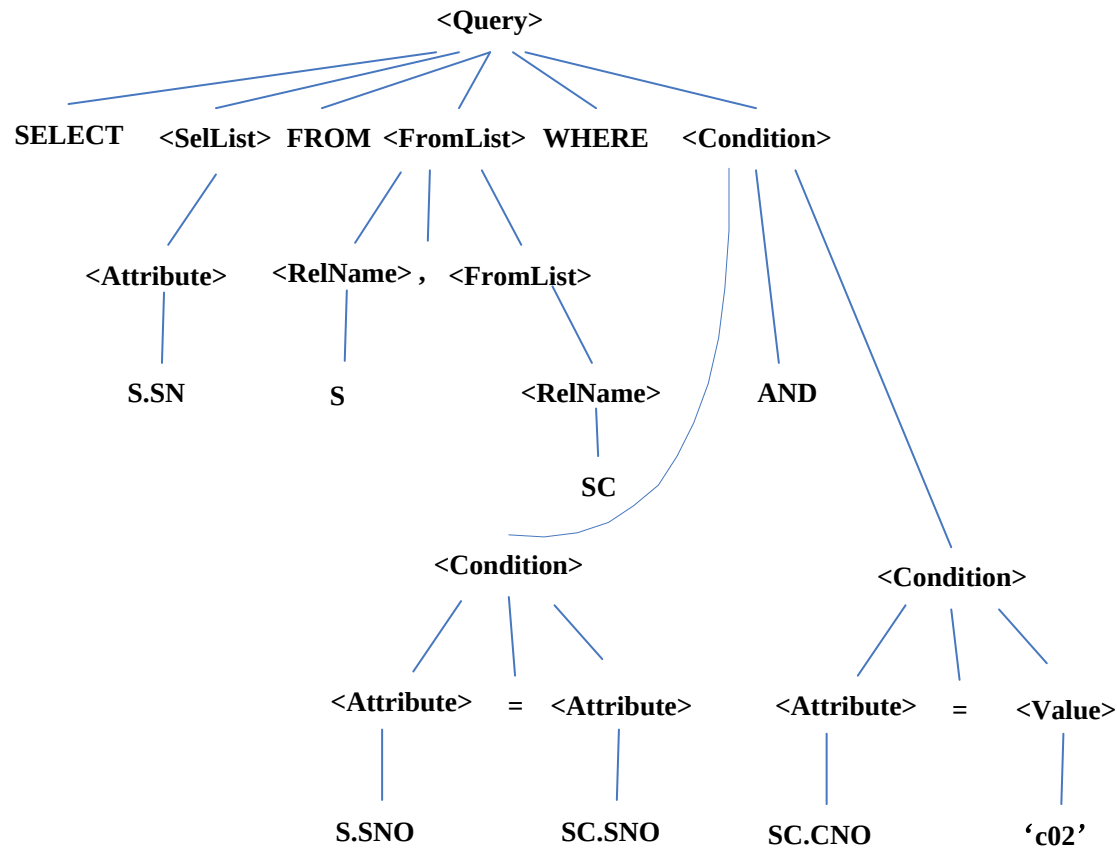
【例】在“学生-课程”数据库中查询选修了课程号为“c02”课程的学生姓名。

```
Q1: SELECT SN
      FROM S, SC
      WHERE S.Sno=SC.Sno AND SC.Cno=' c02 ';
```

```
Q2: SELECT SN
      FROM S
      WHERE Sno IN (SELECT Sno
                     FROM SC
                     WHERE Cno='c02')
```

5.2 数据库查询处理过程

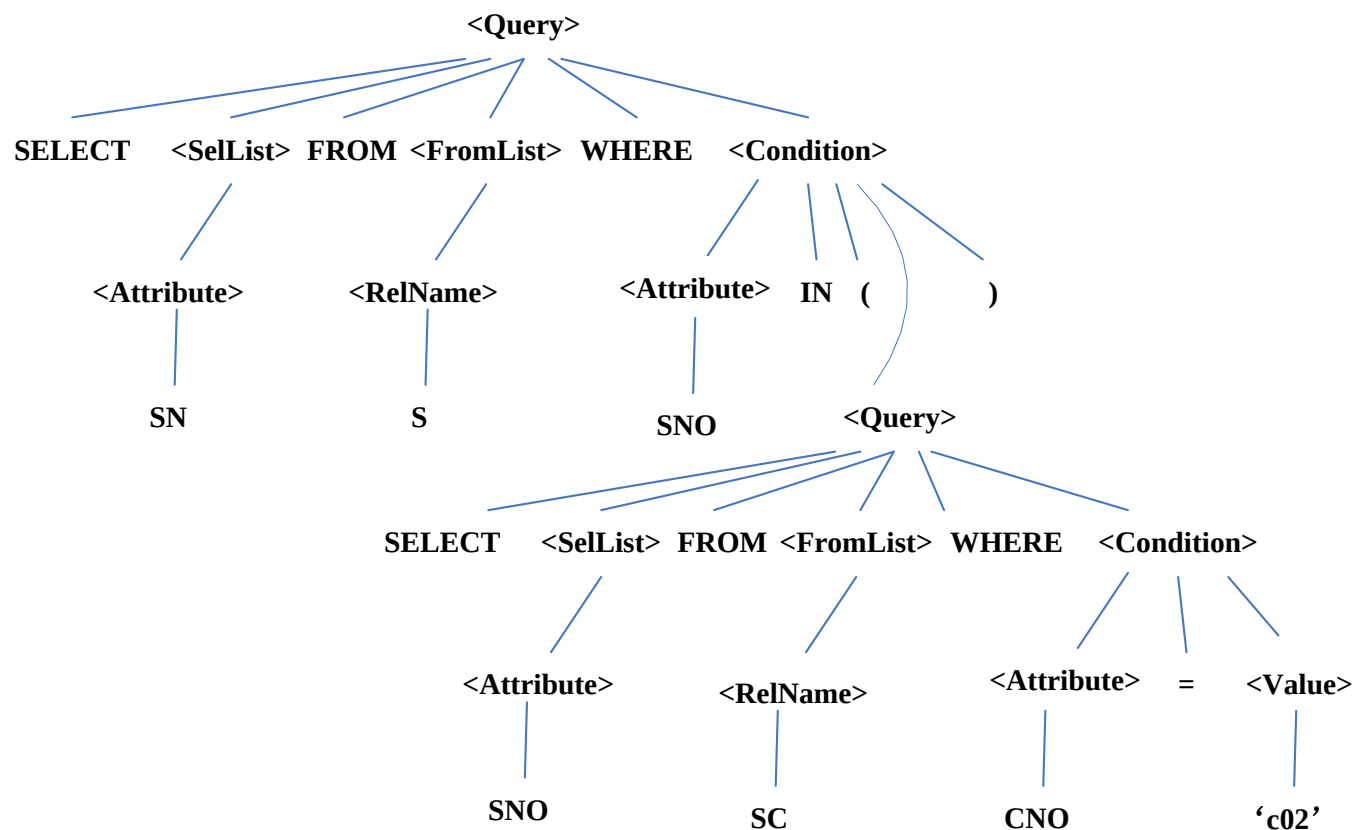
【例】在“学生-课程”数据库中查询选修了课程号为“c02”课程的学生姓名。



用查询语句Q1实现两个关系的连接查询的语法分析树

5.2 数据库查询处理过程

【例】在“学生-课程”数据库中查询选修了课程号为“c02”课程的学生姓名。



用查询语句Q2实现两个关系的嵌套查询的语法分析树

5.2 数据库查询处理过程

- 查询有效性检查

- **根据数据字典对合法的查询语句进行语义检查。**

- 检查语句中的数据库对象在所查询的特定数据库模式中是否为有效且有语义含义的名字。
 - 检查所有属性的类型是否与其使用相对应，以及根据数据字典中的用户权限和完整性约束定义对用户的存取权限进行检查。

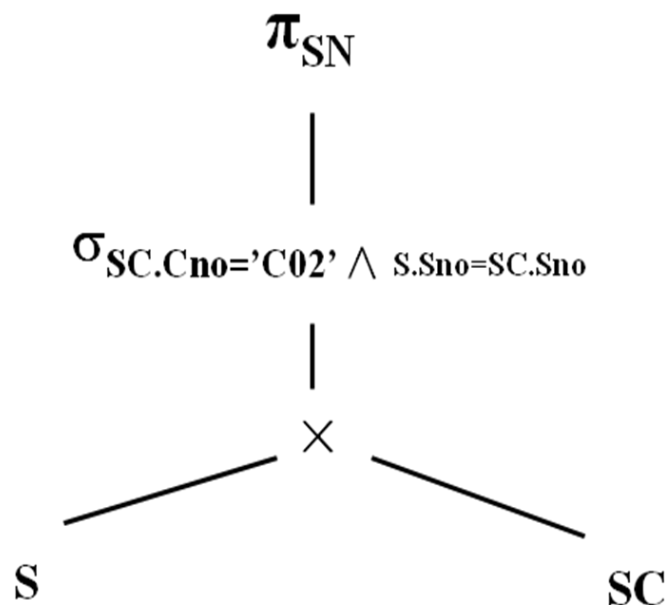
5.2 数据库查询处理过程

- 生成关系代数初始查询树
 - 查询预处理器采用一些相应的规则，用一个或多个关系代数运算符替换语法树上的结点与结构，生成一个对应于SQL查询的关系代数初始查询树。
 - 关系代数查询树是一个树数据结构，在查询树中，查询的输入关系表示为叶结点，关系代数操作表示为内部结点，一元关系操作符只有一个子结点，二元关系操作符有两个子结点。

5.2 数据库查询处理过程

【例】在“学生-课程”数据库中查询选修了课程号为“c02”课程的学生姓名。

Q1: $\pi_{SN} (\sigma_{S.Sno=SC.Sno \wedge SC.Cno='c02'} (S \times SC))$



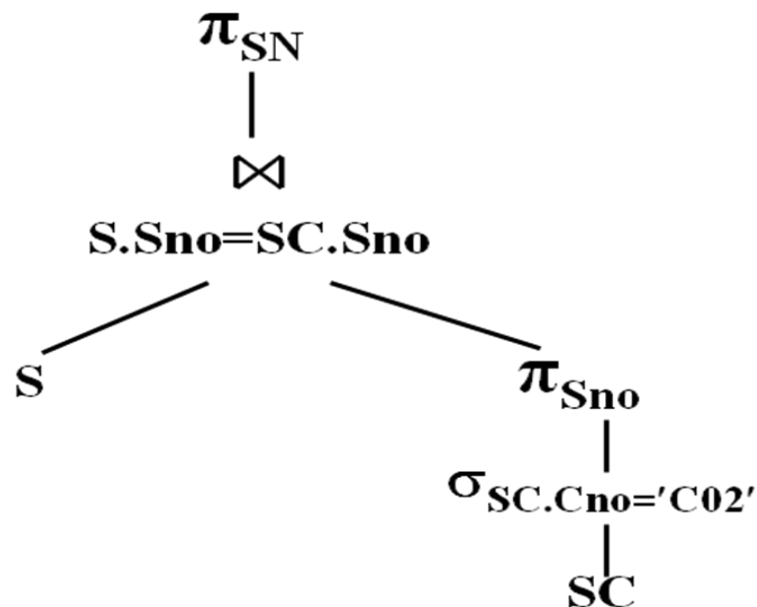
每个内部节点用关系操作符来标记，每个叶子结点用关系名来标记。一元关系操作符只有一个孩子，二元操作符有两个孩子。

Q1查询的关系代数查询树

5.2 数据库查询处理过程

【例】在“学生-课程”数据库中查询选修了课程号为“c02”课程的学生姓名。

Q2: $\pi_{SN} (S \bowtie \pi_{Sno} (\sigma_{SC.Cno='c02'} (SC)))$



Q2查询的关系代数查询树

5.2 数据库查询处理过程

```
select * from table1 where name='zhangsan' and tID > 10000
```

```
select * from table1 where tID > 10000 and name='zhangsan'
```

这两个语句执行是不是一样的？

如果tID是一个聚合索引，那么后一句仅仅从表的10000条以后的记录中查找就行了；而前一句则要先从全表中查找看有几个name='zhangsan'的，而后再根据限制条件条件tID>10000来提出查询结果。

例如SQL SERVER中有一个“查询分析优化器”，它可以计算出where子句中的搜索条件并确定哪个索引能缩小表扫描的搜索空间，也就是说，它能实现自动优化。

虽然查询优化器可以根据where子句自动的进行查询优化，但大家仍然有必要了解一下“查询优化器”的工作原理，如非这样，有时查询优化器就会不按照您的本意进行快速查询。

5.2 数据库查询处理过程

在查询分析阶段，查询优化器查看查询的每个阶段并决定限制需要扫描的数据量是否有用。如果一个阶段可以被用作一个扫描参数（SARG），那么就称之为可优化的，并且可以利用索引快速获得所需数据。

SARG的定义：用于限制搜索的一个操作，因为它通常是指一个特定的匹配，一个值得范围内的匹配或者两个以上条件的AND连接。形式如下：

列名 操作符 <常数 或 变量>

或

<常数 或 变量> 操作符列名

列名可以出现在操作符的一边，而常数或变量出现在操作符的另一边。如：

Name='张三'

价格>5000

5000<价格

Name='张三' and 价格>5000

5.2 数据库查询处理过程

查询执行计划

SQL Server有几种方式查找数据记录

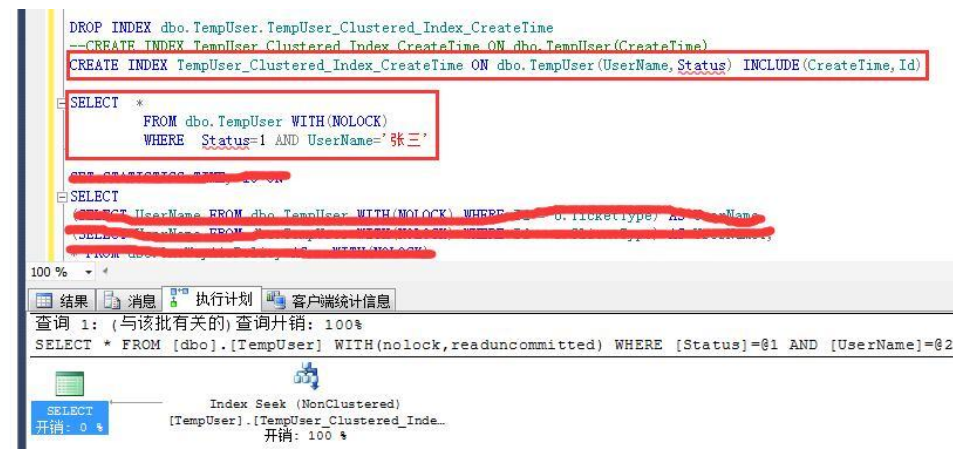
- [Table Scan] 表扫描（最慢），对表记录逐行进行检查
- [Clustered Index Scan] 聚集索引扫描（较慢），按聚集索引对记录逐行进行检查
- [Index Scan] 索引扫描（普通），根据索引滤出部分数据在进行逐行检查
- [Index Seek] 索引查找（较快），根据索引定位记录所在位置再取出记录
- [Clustered Index Seek] 聚集索引查找（最快），直接根据聚集索引获取记录



没有主键的表查询[表扫描]



有主键的表查询[聚集索引扫描]



建立非聚集索引并把其他显示列加入索引中[索引查找]

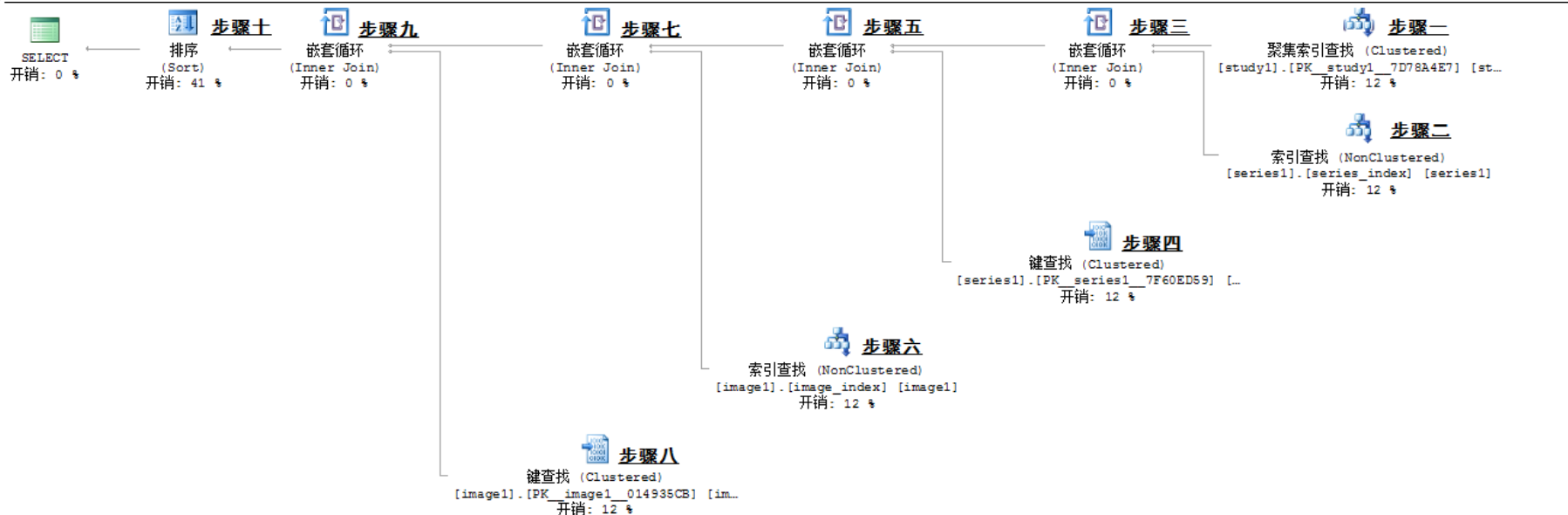
5.2 数据库查询处理过程

查询执行计划

```

1  SELECT <所需列>   FROM study1
2  INNER JOIN series1
3      ON (study1.study_uid_id = series1.study_uid_id) --连接条件1
4  INNER JOIN image1 image1
5      ON (series1.series_uid_id = image1.series_uid_id) --连接条件2
6  where ((study1.user_group &8) != 0) --过滤条件1
7      and (series1.modality) not in ('PR', 'KO', 'SR', 'AU') --过滤条件2
8      and study1.study_uid = 'xxx' --过滤条件3
9  order by <排序列>;
10

```



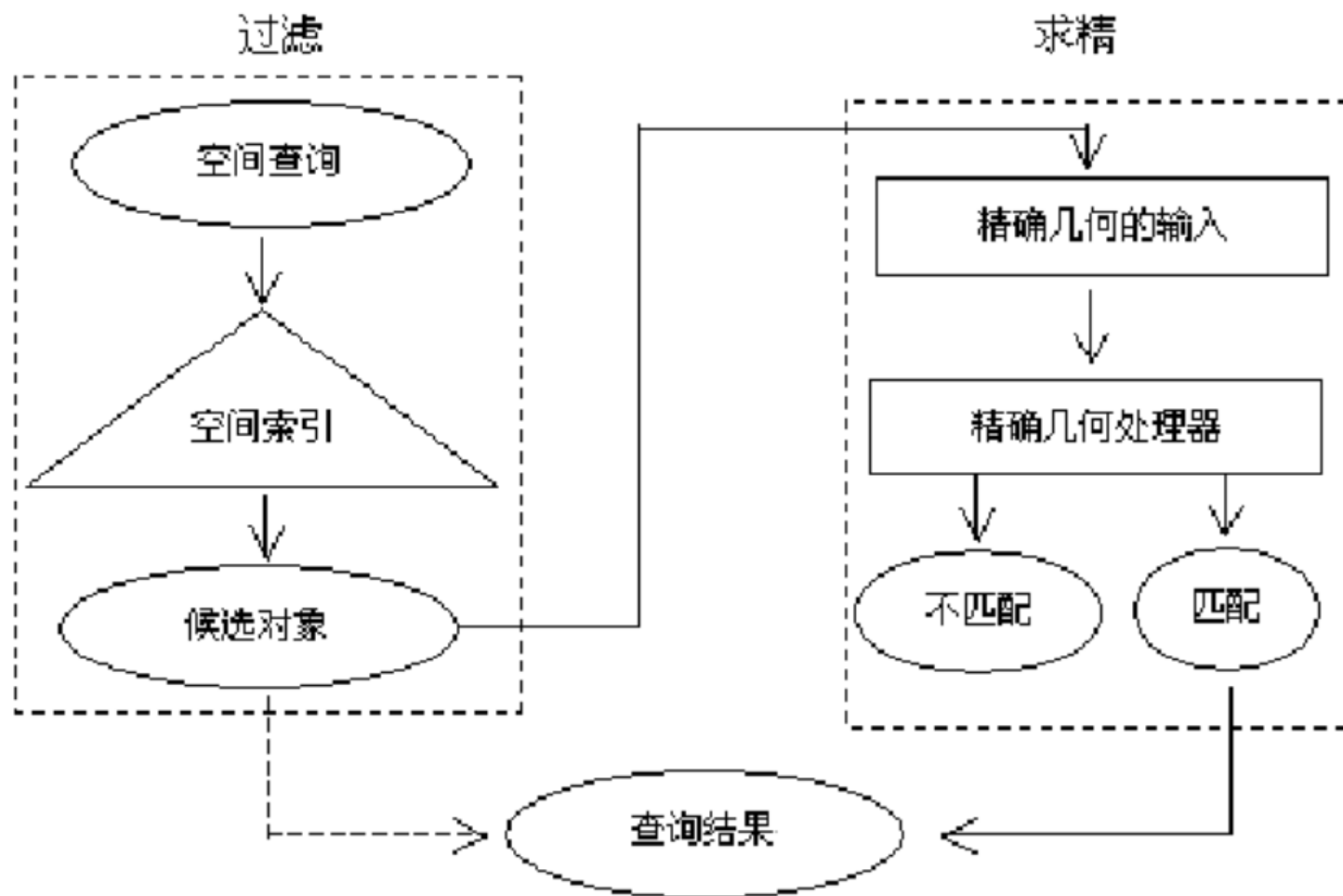
5.3 空间查询执行过程

空间查询处理的过程一般分为两个步骤——过滤和求精。

当执行空间查询时，查询处理器首先访问空间索引。空间索引中存储的是空间对象的近似描述，对这些近似描述做相关的操作，排除掉不可能满足查询条件的对象，余下的对象可能满足查询条件，这些对象构成了候选集。这就是过滤步骤。将候选集对象的实际数据输入求精步进行下一步的处理。

在求精步，需要测试候选对象是否确实满足查询条件(由空间谓词描述)，采用复杂的计算几何算法，这个测试由不同的阶段组成。

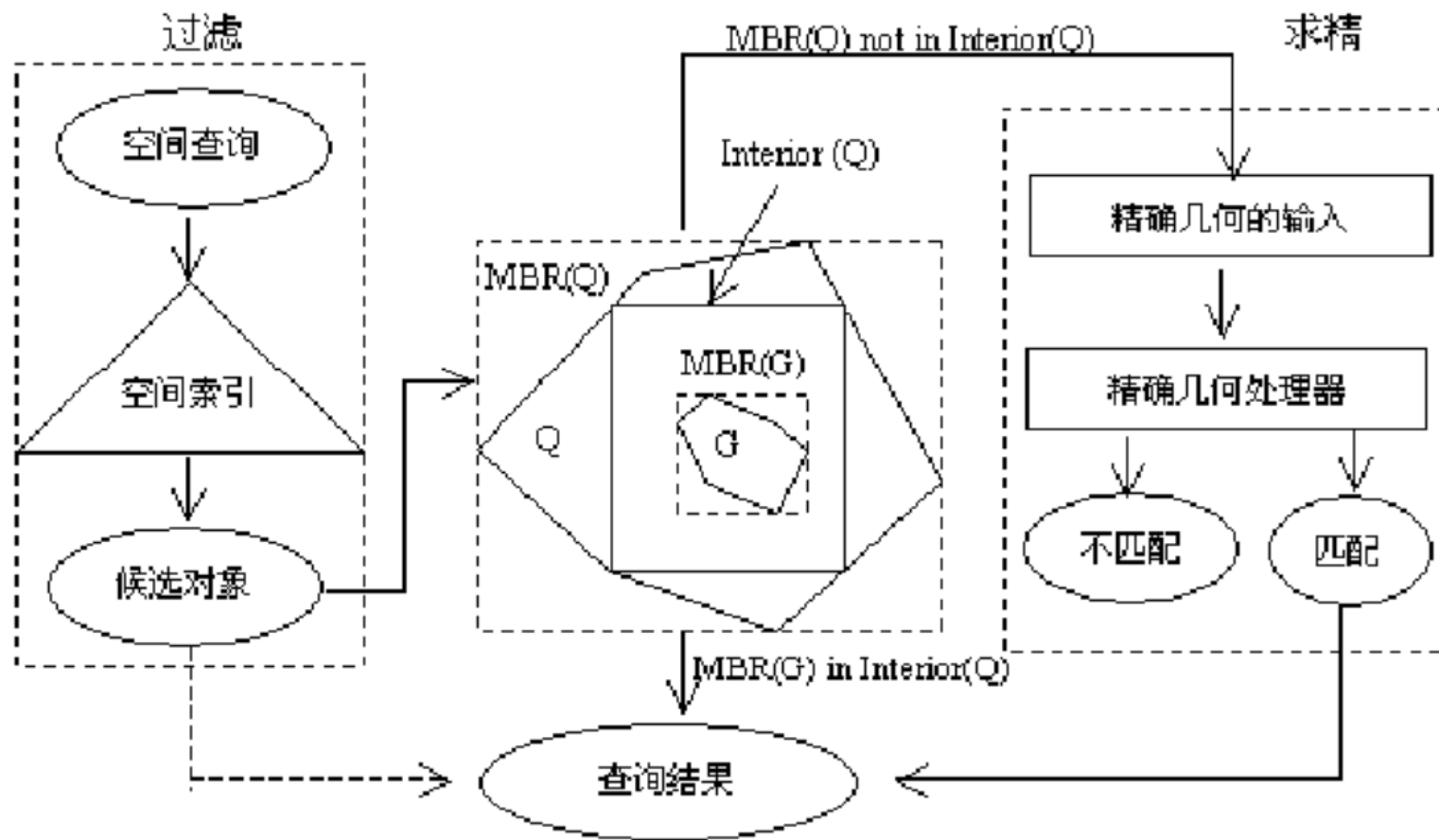
经过求精步的测试后，满足条件的对象作为最终的结果输出。



5.3 空间查询执行过程

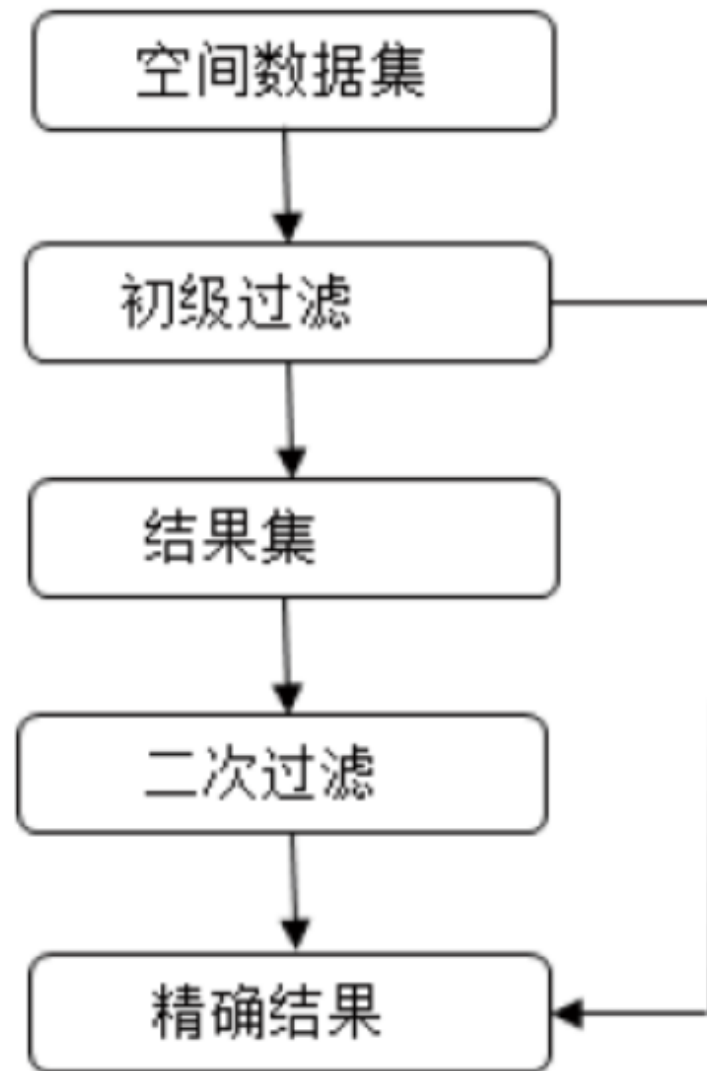
在过滤步和求精步之间插入一个中间步骤，对对象的 MBR 与查询窗口的内部表示(如图中的 Interior(Q))的关系进行判断，满足查询条件(如图中的 MBR(G)完全包含在 Interior(Q)内)的直接作为结果输出，否则仍进行计算，但计算量已大大减少。

此项研究已应用于 Oracle 空间数据库。



5.3 空间查询执行过程

Oracle Spatial 使用双层查询模型来解决空间查询问题, 即的一级过滤操作和二级过滤操作. 经过两次过滤, 将返回精确的查询结果集。



目录

CONTENTS

6

空间查询优化

6.1 为什么需要查询优化

6.2 关系数据库查询优化

6.3 空间查询优化

6.1 有什么需要查询优化

- **查询优化的必要性**

- **数据的物理独立性，即数据的存取路径、存储结构、存取策略对用户透明，查询效率不如非关系数据库。**
- **查询优化极大地影响RDBMS的性能。**
- **查询优化并不只是DBMS的任务，用户的查询计划质量直接影响优化效率和结果，因此部分用户有必要了解查询优化的概念和相关技术，写出‘好’的查询，执行效率高的语句。**

- **查询优化的可能性**

- **关系数据语言的级别很高，使DBMS可以从关系表达式中分析查询语义。**

6.1 有什么需要查询优化

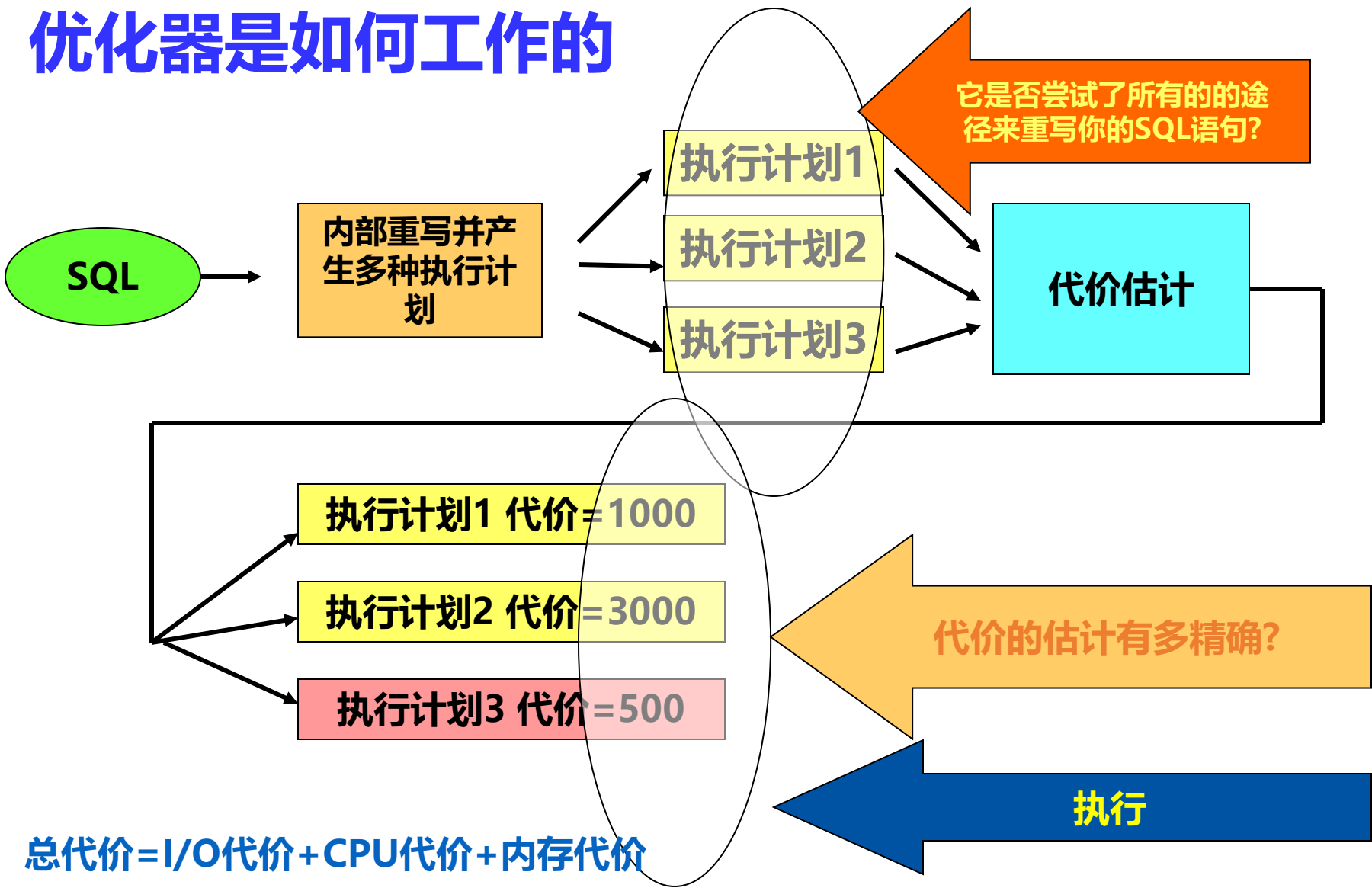
由DBMS进行查询优化的好处

用户不必考虑如何最好地表达查询以获得较好的效率。系统可以比用户程序的优化做得更好

- **优化器**可以从数据字典（DD）中获取许多统计信息，而用户程序则难以获得这些信息。
- 如果数据库的物理统计信息改变了，系统可以自动对查询**重新优化**以选择相适应的执行计划。在非关系系统中必须重写程序，而重写程序在实际应用中往往是不太可能的。
- 优化器可以考虑数百种不同的执行计划，而程序员一般只能考虑有限的几种可能性。
- 优化器中包括了很多复杂的优化技术

6.1 有什么需要查询优化

优化器是如何工作的



6.2 关系数据库查询优化

• 查询优化的目标

选择有效策略，求得给定关系表达式的值

• 查询优化步骤

1. 将查询转换成某种内部表示，通常是**语法树**。
2. 根据一定的等价变换规则把语法树转换成**标准**

[工具]：关系代数等价变换规则、关系代数表达式的优化算法

3. 选择低层的操作算法

对于语法树中的每一个操作计算各种执行算法的执行代价，选择代价小的执行算法

4. 生成查询计划(查询执行方案)

查询计划是由一系列内部操作组成的。

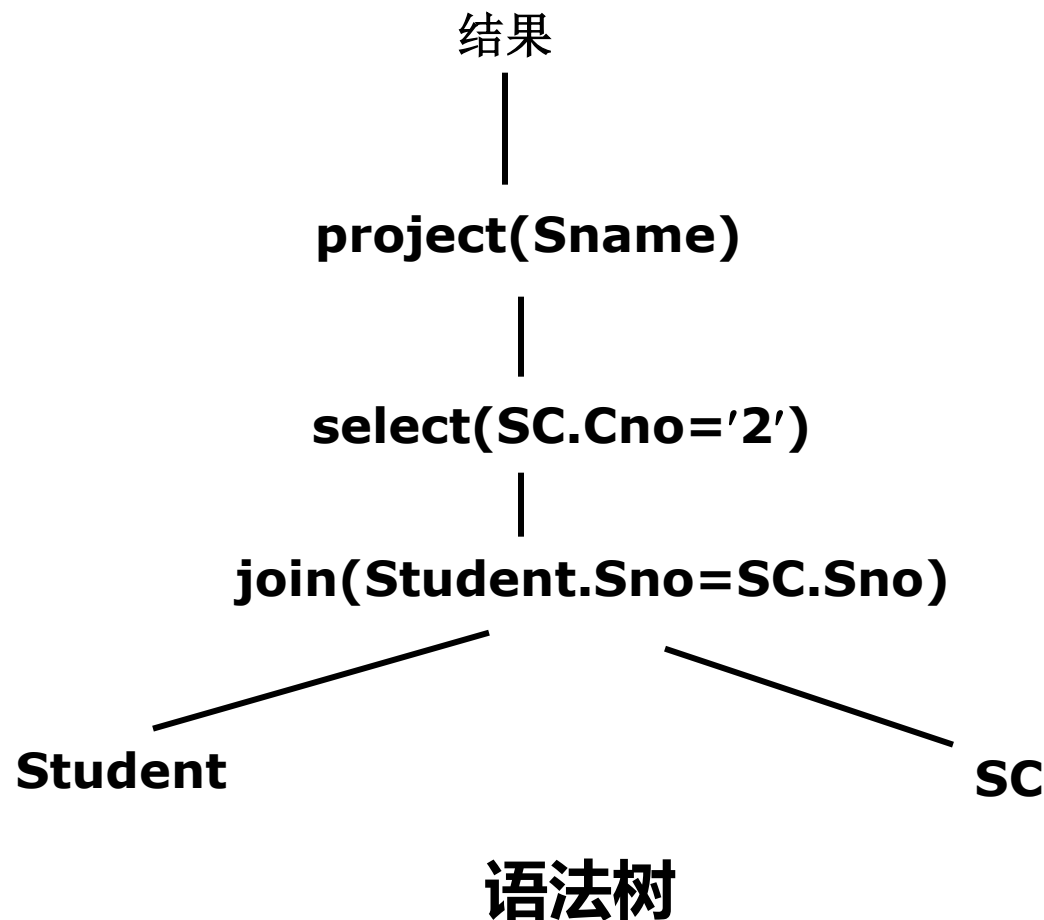
6.2 关系数据库查询优化

• 查询优化的一般步骤

(1) 把查询转换成某种内部表示

例：求选修了课程 C 2 的学生姓名

```
SELECT Student.Sname  
FROM Student, SC  
WHERE Student.Sno=SC.Sno  
AND SC.Cno='2';
```



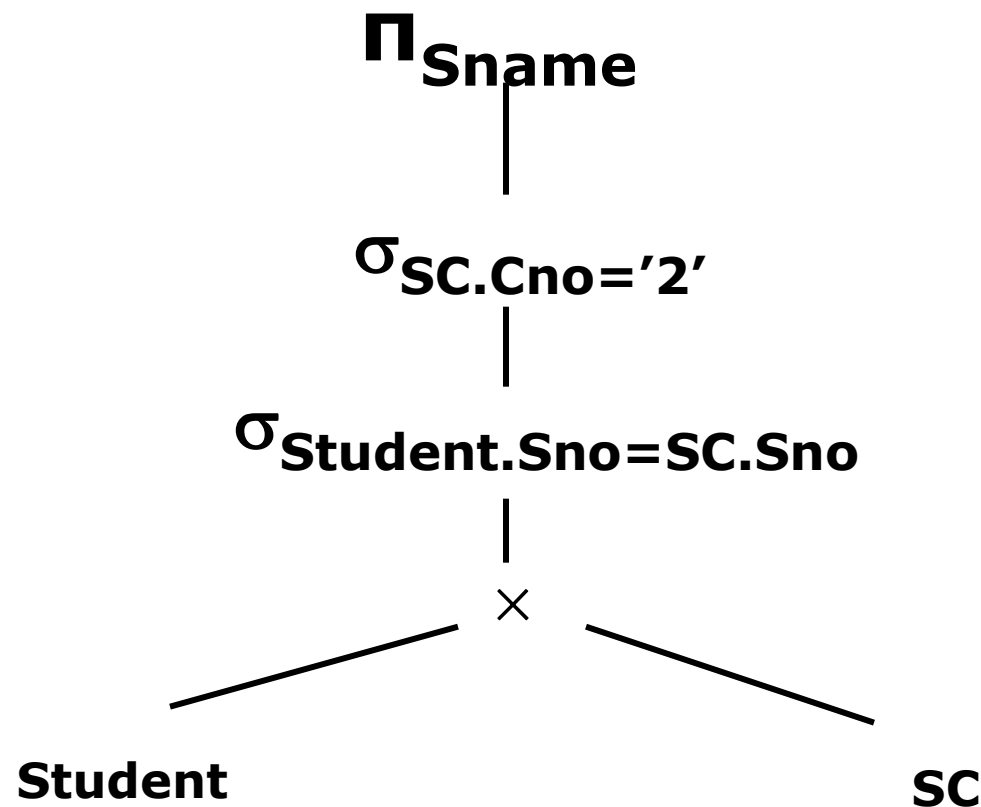
6.2 关系数据库查询优化

• 查询优化的一般步骤

(1) 把查询转换成某种内部表示

例：求选修了课程 C 2 的学生姓名

```
SELECT Student.Sname
FROM Student, SC
WHERE Student.Sno=SC.Sno
AND SC.Cno='2';
```



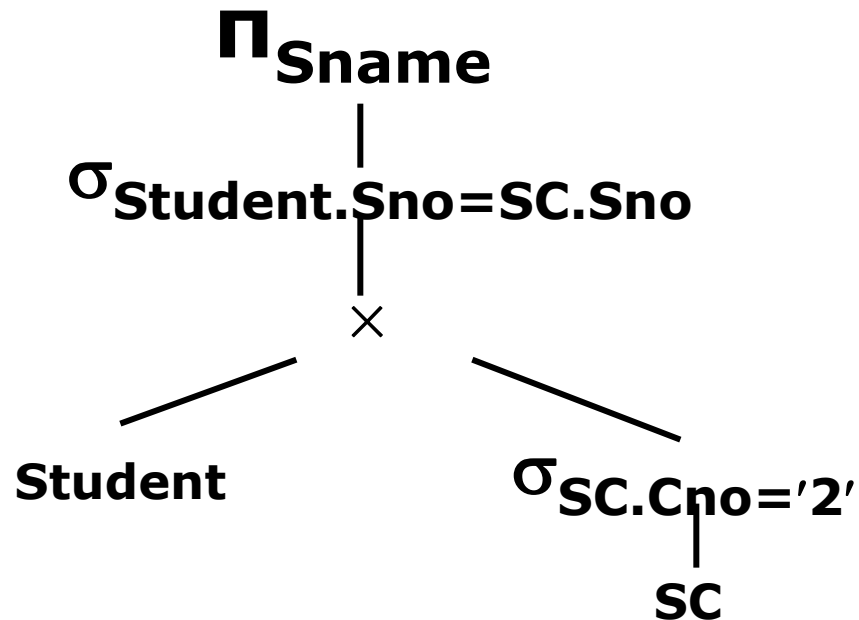
关系代数语法树

6.2 关系数据库查询优化

• 查询优化的一般步骤

(2) 代数优化

利用优化算法把语法树转换成标准（优化）形式



6.2 关系数据库查询优化

• 查询优化的一般步骤

(3) 物理优化：选择低层的存取路径

- 优化器查找数据字典获得当前数据库状态信息

- 选择字段上是否有索引
- 连接的两个表是否有序
- 连接字段上是否有索引

- 然后根据一定的优化规则选择存取路径

如本例中若SC表上建有Cno的索引，则应该利用这个索引，而不必顺序扫描SC表。

6.2 关系数据库查询优化

• 查询优化的一般步骤

(4) 生成查询计划，选择代价最小的

- 在作连接运算时，若两个表(设为R1, R2)均无序，连接属性上也没有索引，则可以有下面几种查询计划：
 - 对两个表作排序预处理
 - 对R1在连接属性上建索引
 - 对R2在连接属性上建索引
 - 在R1, R2的连接属性上均建索引
- 对不同的查询计划计算代价，选择代价最小的一个。
- 在计算代价时主要考虑磁盘读写的I/O数，内存CPU处理时间在粗略计算时可不考虑。

6.2 关系数据库查询优化

查询调优：分为四个层级

语法级：查询语言层的优化，基于语法进行优化

代数级：查询使用形式逻辑进行优化，运用关系代数的原理进行优化

语义级：根据完整性约束，对查询语句进行语义理解，推知一些可优化的操作

物理级：物理优化技术，基于代价估算模型，比较得出各种执行方式中代价最小的路径

6.2 关系数据库查询优化

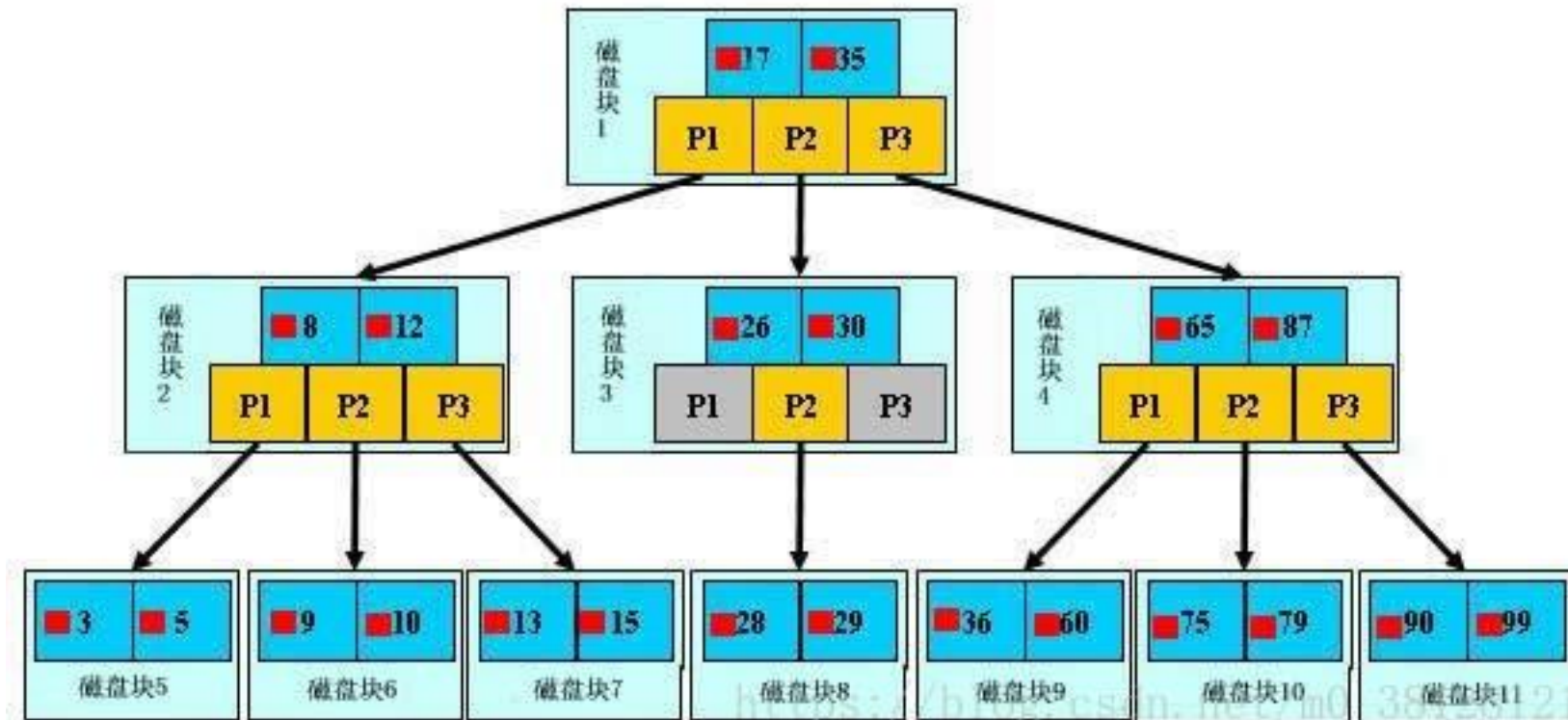
优化思路：

- 一：将过程性查询转换为描述性的查询，如视图重写（后更试图重写技术）
- 二：将复杂的查询（如嵌套子查询，外连接（全外，左外，右外），嵌套连接）尽可能的转换为多表连接查询
- 三：将效率低的谓词转换为等价的效率高的谓词（如等价谓词重写）
- 四：利用等式和不等式，简化where,having和ON条件

6.2 关系数据库查询优化

1.使用索引优化查询

应尽量避免全表扫描，首先应考虑在 where 及 order by ,group by 涉及的列上建立索引



6.2 关系数据库查询优化

2. 优化 SQL 语句

通过 `explain`(查询优化器)用来查看 SQL 语句的执行效果

可以帮助选择更好的索引和优化查询语句， 写出更好的优化语句。

通常我们可以对比较复杂的尤其是涉及到多表的 `SELECT` 语句， 把关键字 `EXPLAIN` 加到前面， 查看执行计划。

例如： `explain select * from news;`

4.2 关系数据库查询优化

2.优化 SQL 语句

尽量不使用select * from table这种方式

嵌套多级子查询的复杂SELECT 语句,可以考虑适当拆成几步,先生成一些临时数据表,再进行关联操作

在WHERE 语句中, 尽量避免对索引字段进行计算操作

避免在WHERE子句中使用in, not in, or 或者having

避免使用耗费资源的操作, 带有STINCT,UNION,MINUS,INTERSECT,ORDER BY 的SQL语句会启动SQL引擎 执行耗费资源的排序(SORT)功能

.....

6.2 关系数据库查询优化

3 优化数据库对象

3.1 优化表的数据类型

使用 `procedure analyse()` 函数对表进行分析，该函数可以对表中列的数据类型提出优化建议。能小就用小。表数据类型第一个原则是：使用能正确的表示和存储数据的最短类型。这样可以减少对磁盘空间、内存、cpu 缓存的使用。

使用方法： `select * from 表名 procedure analyse();`

6.2 关系数据库查询优化

3.2 对表进行拆分

通过拆分表可以提高表的访问效率。 有 2 种拆分方法

1.垂直拆分

把主键和一些列放在一个表中， 然后把主键和另外的列放在另一个表中。 如果一个表中某些列常用， 而另外一些不常用， 则可以采用垂直拆分。

2.水平拆分

根据一行或者多列数据的值把数据行放到二个独立的表中。

3.3 使用中间表来提高查询速度

创建中间表， 表结构和源表结构完全相同， 转移要统计的数据到中间表， 然后在中间表上进行统计， 得出想要的结果。

6.2 关系数据库查询优化方法

4. 硬件优化

4.1 CPU 的优化：选择多核和主频高的 CPU。

4.2 内存的优化：使用更大的内存。 将尽量多的内存分配给 MYSQL 做缓存。

4.3 磁盘 I/O 的优化

4.3.1 使用磁盘阵列：RAID 0+1 是 RAID 0 和 RAID 1 的组合形式， 它在提供与 RAID 1 一样的数据安全保障的同时， 也提供了与 RAID 0 近似的存储性能。

4.3.2 调整磁盘调度算法：选择合适的磁盘调度算法， 可以减少磁盘的寻道时间

6.3 空间查询优化

空间查询的优化过程大致分为如下4个阶段:

(1) 将查询转换为某种内部表示

其一般是将查询条件用语法树的形式表示.

(2) 进行某些提高效率的表达式变换

如,先进行选择再连接,先进行投影再连接等的变换,而且变换规则通常是有正确性保证的.

(3) 谓词代价计算

(4) 选择代价最小的执行计划

6.3 空间查询优化

① 空间索引

这是从数据存取的角度来优化空间查询。由于空间数据量庞大，有效读写海量数据是解决空间数据库瓶颈的重要方法。

② 查询处理算法

这是从提高具体查询执行效率的角度来优化空间查询。通过尽可能少的复杂几何计算就能得到正确的查询结果、提高查询执行的效率是优化查询处理算法的主要目标。

③ 代价模型

这是从指导查询过程（即采用哪一种查询计划）的角度来优化空间查询。这一过程在处理含有多个空间查询条件的查询时起到了重要的作用。

6.3 空间查询优化

基于空间索引的空间查询优化技术

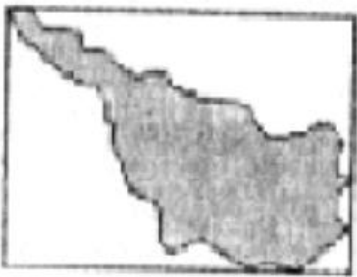
R-树索引结构简单，能对空间对象进行有效地处理，故可提出一些基于 R-树的算法来优化空间连接这一复杂耗时的空间查询操作。这些算法要求每个参与空间连接的对象集合均预先建立了 R-树索引。

R-树的匹配算法是直接的，它从匹配两个 R-树的根节点开始，寻找是否有重叠的情况，然后深度优先递归搜索匹配的子节点，当达到叶节点时，得出匹配结果并输出。

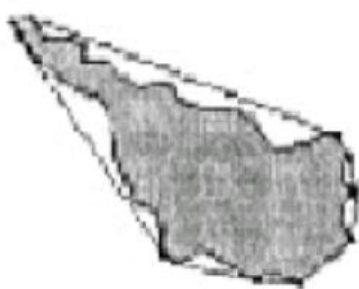
6.3 空间查询优化

在过滤步骤中的对象近似应当是简单的，这样才能够实现快速的过滤。另一方面，对空间对象的近似程度越高，滤除未命中对象的效果越好。一般来说，用少量参数表示的凸近似是合适的。在过滤步骤中通常采用 MBR 近似，其近似效果有限。

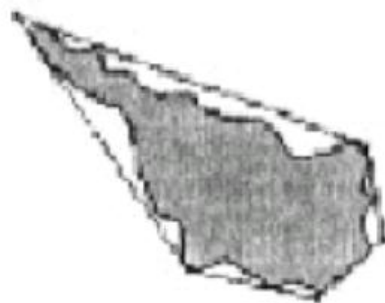
与MBR 相比，这些近似描述的 SAM(空间存取方法)需要访问更多的数据页，但由于它们的近似质量高，可以减少未命中对象，从而减少后继处理中的几何计算开销。显然，近似使用的参数越多，近似质量越高。



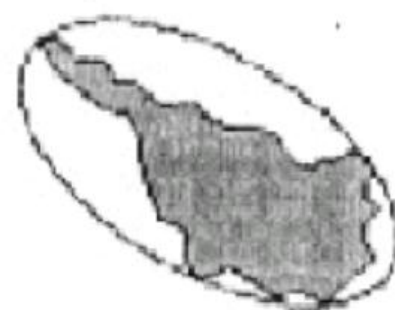
最小边界矩形



凸壳



五角形



椭圆

谢谢大家!

